#14 캠 정지 — 1분 큰 그림 + 셀프 점검

출처: Notion `2차프로젝트 → [#14] 코드리뷰 · 캠 정지` (PR #116)

1분 큰 그림 (중학생 버전)

상황: 켜져 있는 카메라를 끌 수 있어야 한다. 준비 중인 카메라도 중간에 취소할 수 있어야 한다. (#13 가동의 반대)

해결 (#14): 관리자가 "정지" 버튼을 누르면 분석 서버에 **"분석을 멈춰"** 라고 시키고, 카메라를 **정지 상태(STOPPED)** 로 저장하는 API를 만들었다.

#13과 가장 다른 점: 분석 서버에 말이 **전달되지 않아도** 우리 쪽은 **정지로 저장**하고 "분석 서버에 전달 실패"라는 **경고만** 돌려준다. (#13 가동은 실패하면 ERROR와 오류 응답을 줬다.)

비유 (교실 스위치 끄기): 먼저 "이 교실은 앞으로 켜지 않는다"는 메모(`run_requested = 0`)를 확정하고, 관리실(분석 서버)에 "끄세요"라고 전화한다. 전화가 안 걸려도 **우리 장부에는 "꺼짐"으로 적는다.** 메모가 있어서 전화가 다시 연결돼도 이 교실은 다시 켜지지 않는다.

서버가 하는 일 3단계

1. 카메라가 있고 삭제되지 않았는지 확인하고 `run_requested = 0`을 **먼저 확정**한다. (없으면 404)
2. 분석 서버에 `stop`을 호출한다. (**15초** 안에 답이 없으면 포기) 준비 중이면 분석 서버가 준비를 취소한다.
3. 카메라를 **STOPPED, 진행률 0**으로 저장하고 이전 오류 내용을 지운다.

응답: `200 {"status": "STOPPED", "warning": null}` 또는 전달에 실패하면 `"warning": "분석 서버에 전달 실패"`

#13 가동 vs #14 정지

	#13 가동	#14 정지
분석 서버 호출이 실패하면	**ERROR** + 오류 응답(502/422)	**STOPPED** + 경고(200)
`run_requested`	1로 만듦	0으로 만듦
타임아웃	5초	15초 (분석 서버가 종료를 최대 10초 기다림)
상태 확인	STOPPED/ERROR만 가능 (아니면 409)	**제한 없음** (이미 정지된 캠에도 가능)
트랜잭션	세 단계로 쪼갬	세 단계로 쪼갬 (같은 구조)

왜 정지는 실패해도 STOPPED로 저장하나? 관리자는 "끄고 싶다"는 뜻을 분명히 밝혔다. 분석 서버가 응답하지 않는다고 우리 장부에 "켜짐"으로 남겨 두면 관리자 입장에서 끌 수 없는 카메라가 된다. 그래서 우리 쪽은 끄고, 분석 서버에는 못 전했다고 알려 준다. (명세가 정한 것이다.)

핵심 개념 4가지

① 왜 `run_requested = 0`을 전화보다 먼저 확정하나?

전화가 실패하거나 앱이 중간에 죽어도 "켜지 말라"는 기록이 남아야, 분석 서버가 나중에 다시 켜져도 이 카메라를 **다시 가동하지 않기** 때문이다. 테스트(`stop_cancelsRunRequestBeforeCallingAnalyzer`)는 분석 서버가 요청을 **받는 순간**에 DB를 읽어서 `run_requested`가 이미 0인지 확인한다.

② 트랜잭션을 세 단계로 쪼갠 이유 (#13과 같음)

분석 서버를 최대 15초 기다리는 동안 DB를 붙들지 않기 위해서다. 그리고 호출 결과와 상관없이 앞(① 메모)과 뒤(③ 정지 기록)가 **각각 확정**돼야 한다.

③ 이미 정지된 카메라에도 정지를 시켜도 안전하다 (멱등)

멱등은 "몇 번 해도 결과가 같다"는 뜻이다. 분석 서버의 정지가 멱등이라 두 번 불러도 괜찮다. 오히려 상태가 엇갈린 카메라(우리 앱은 "정지", 분석 서버는 "가동 중")를 정지로 **정리하는 길**도 된다.

④ `AnalyzerClient.stopCamera`를 건드리지 않는다

팀원 #15가 같은 메서드를 쓰고 있다. 던지는 예외 종류(`RestClientException`)를 바꾸면 그쪽이 깨질 수 있어서 그대로 두었다.

반드시 알아야 할 것 두 가지

1) 명세의 처리 순서 4·5·6이 빠졌다.

순서	내용	못 한 이유
4	더미 캠의 마지막 녹화 `ended_at` 채우기	recording 엔티티가 팀원 #17(PR #98)에만 있다
5	열린 이상 이벤트 종료	이벤트 테이블이 어디에도 없다
6	작업 기록(CAMERA_RUN, op: STOP)	#6의 PR #113이 열려 있지만 병합 전이다

그래서 **이 PR만으로는 이슈의 완료 기준("명세의 처리 순서대로")을 다 채우지 못한다.** 코드에는 호출 자리만 주석으로 남겼다. 또 #113의 작업 기록 서비스는 **트랜잭션 없이 부르면 거부**하므로, 나중에 연결할 때는 3단계(정지 기록) 트랜잭션 **안에서** 호출해야 한다.

2) 실제 분석 서버와 연결하지 못했다. 이슈 완료 기준 "준비 중 정지 시 준비 취소 확인" 중 이 PR은 "앱이 정지를 지시하고 STOPPED로 저장한다"까지만 확인한다. 준비를 **실제로 멈추는 일은 분석 서버(flask #2) 몫**이다.

솔직하게 말해야 할 약점

1. **처리 순서 4·5·6이 없다.** 녹화 종료·이벤트 종료·작업 기록. (R1)
2. **실제 분석 서버와 연결 못 했다.** 준비 취소는 분석 서버 몫이라 앱 쪽까지만 확인했다. (R2)
3. **권한 검사가 없다.** 보안 설정이 dev에 없다. (R3)
4. **분석 서버의 모든 오류 응답을 "전달 실패" 경고 하나로 합쳤다.** 401(토큰 설정 오류), 5xx, 404 모두 같은 문구라서 관리자가 원인을 구분할 수 없다. 내가 정한 것이다. (R4)

5. **분석 서버가 응답하지 않으면 정지 요청이 최대 15초** 걸린다. 그동안 DB엔 `run_requested=0`이지만 `status`는 이전 상태다. (R5)
6. **15초 타임아웃 테스트가 약 15초 걸려** CI 시간이 늘어난다. (R9)
7. **마이그레이션 V5가 #113과 겹친다.** #113이 `work_history`를 V5로 만들고 camera도 V5라서 번호를 정리해야 한다. (속성 요약에 적힌 확인 사항)

셀프 점검 (자기 말로 답해 보기)

Q1. 이 API는 뭘 하는 건가요? #13 가동과 가장 다른 점은 뭔가요?

답:

Q2. 분석 서버에 전달이 실패해도 왜 우리 쪽은 STOPPED로 저장하나요?

답:

Q3. `run_requested = 0`을 분석 서버에 전화하기 전에 먼저 확정하는 이유는요?

답:

Q4. 이 PR에서 구현하지 못한 것 3가지는 뭐고 왜 못 했나요?

답:

내일 반드시 말할 수 있어야 하는 4가지

1. 이 API가 뭘 하고 #13과 뭐가 다른가 (실패해도 STOPPED + 경고)
2. `run_requested=0`을 먼저 확정하는 이유 (분석 서버가 다시 켜져도 가동 안 함)
3. 트랜잭션 세 단계 (분석 서버를 기다리는 동안 DB를 안 붙듦, 앞뒤 각각 확정)
4. 약점: 처리 순서 4·5·6 미구현(선행 PR 대기), 실제 분석 서버 미연결, 오류 응답을 한 경고로 합침